

UCE: Unified Change Enforcement for Governed LLM Coding Agents

Preet Patel

Computer Science, University of Illinois Chicago
Chicago, IL, USA — patelpr@uic.edu

Abstract—Production deployments of LLM coding agents lack a reproducible layer for blast-radius analysis, requirement traceability, and role-based access control (RBAC). We present UCE (Unified Change Enforcement): a deterministic Neo4j knowledge graph built from source, schema, and governance artifacts, exposed to agents via the Model Context Protocol (MCP), and coupled to a pre-merge enforcement gate. We report a multi-repository study (114 destructive-change scenarios; TalkAI, Melodi, Expenses, Spark) with an *independent* evaluation oracle based on a from-scratch TypeScript import resolver and governance-document parsing—never reading UCE’s Cypher output. Primary result (enforcement): editor-role Claude Sonnet 4.5 self-catches only 7.0% of governance/impact issues; UCE’s gate fires on 100% of scenarios where the agent under-declares impact (0.9% false-gate rate; median 55 files missed per TalkAI scenario). Secondary result (MCP planning): under deployment-neutral prompts, agents with `impact_analysis` substantially improve plan recall vs prompt-only baselines; MCP tool output alone matches the oracle at $\sim 98\%$ recall, separating *tool correctness* from *plan adherence*. Context-augmentation baselines (governance docs, inventories, RAG) plateau below graph-backed recall; deterministic RBAC yields 0% breaches vs 4.9% for zero-shot Claude on adversarial rules.

Index Terms—LLM agents, MCP, software governance, impact analysis, knowledge graphs, RBAC, program analysis

I. INTRODUCTION

Autonomous coding agents can edit thousands of files per hour, yet engineering organizations still rely on human reviewers to answer three questions before merging: *What else breaks? Which requirements are violated? Is this principal allowed to touch that path?* Test suites answer breakage only after the fact; prompts answer governance inconsistently.

Hypothesis. Governed change is fundamentally *structured*: dependencies form graphs; policies attach to schema symbols; RBAC is a deterministic predicate. LLMs should *consume* evidence via tools, not reconstruct graphs from truncated context.

UCE ingests repositories into Neo4j, serves MCP tools (`impact_analysis`, `explain_change`, RBAC-gated writes), and compares agent plans against graph impact before allowing edits. Our evaluation deliberately separates three layers:

- 1) **Tool layer**—does MCP return the right blast radius?
- 2) **Plan layer**—does the agent declare that radius in JSON?
- 3) **Gate layer**—does enforcement block under-specified or policy-violating plans?

Layers (2) and (3) decouple “the model saw the answer” from “the organization is protected.”

Contributions.

- Non-circular multi-repo harness (independent import oracle + governance parser).
- Enforcement study showing frontier agents are insufficient without a deterministic gate.
- MCP deployment study under neutral prompts (no verbatim-copy instruction leakage).
- Product fix and analysis: top-level `affected_files` must expose full transitive closure for table/column entities.

II. RELATED WORK

Software engineering agents. SWE-bench [1] and SWE-agent [2] demonstrate issue-level patching with tool-augmented loops; RepoBench [3] stresses repository-scale context. UCE is complementary: *pre-merge* governance and impact evidence rather than post-hoc test passage.

Reasoning agents. ReAct [4] and Reflexion [5] interleave reasoning and action; chain-of-thought [6] improves single-shot reasoning. UCE tools are not learned policies—they are auditable, versioned APIs (MCP [7]) suitable for regulated workflows.

Graphs and code. Graph-RAG [8] and CodexGraph [9] connect LLMs to graph stores for multi-hop retrieval. UCE specializes edge types for schema usage and governance (GOVERNS, ENFORCES) and targets agent *enforcement*, not summarization.

Retrieval baselines. RAG [10] and file-chunk retrieval are standard agent context strategies. Our context ladder shows recall saturates far below import-closure analysis on real repos.

Architecture and policy. Architecture knowledge [11] and policy-as-code motivate explicit, machine-checkable rules. UCE’s RBAC evaluator is the specification; LLM “permission beliefs” are untrusted inputs.

III. PROBLEM AND SYSTEM MODEL

Repository $\mathcal{R}$ comprises files $\mathcal{F}$, schema $\mathcal{S}$, governance $\mathcal{G}$, and RBAC rules $\mathcal{B}$. Change request c targets entity $e \in \mathcal{S} \cup \mathcal{F}$.

Independent oracle $B^{(e)} \subseteq \mathcal{F}$ from import-graph reverse reachability (seeded by schema-aware table/column/file rules). UCE graph yields $B_{\text{uce}}(e)$; MCP returns compact payload with `blast_radius_files`. Agent plan $\pi = (F_\pi, R_\pi, P_\pi, \rho_\pi)$.

Gate. Reject π if $B_{\text{uce}}(e) \not\subseteq F_\pi$, governance sets omit graph findings, or ρ_π claims allow when RBAC denies.

Unified Change Enforcement (UCE)

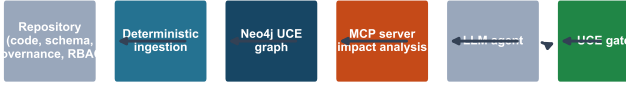


Fig. 1. UCE architecture: deterministic graph, MCP evidence, stochastic agent, deterministic gate.

Metrics. Plan recall $|F_\pi \cap B|/|B|$. Tool recall $|B_{\text{tool}} \cap B|/|B|$. Tool adherence $|F_\pi \cap B_{\text{tool}}|/|B_{\text{tool}}|$. Catch rate: fraction of scenarios where gate fires and $B \not\subseteq F_\pi$.

IV. UCE DESIGN

A. Ingestion

UCE parses ESM imports (including path aliases), builds IMPORTS, links USES_TABLE via import-aware seeding, ingests Drizzle/SQL schemas, and connects requirements/policies to tables and columns. Ingestion is deterministic—identical inputs produce identical edges.

B. MCP tools

`impact_analysis(entity_type, entity_name)` returns the union of direct, transitive, and call-chain file sets at top-level `affected_files`, plus governance fields. Evaluations pass compact JSON with `blast_radius_files` to agents.

C. Enforcement gate

After the agent submits π , UCE recomputes impact and RBAC on the same entity. The gate fires on file omission, silent requirements, or permission breaches—even if the agent invoked MCP tools.

V. EVALUATION METHODOLOGY

Repositories. Four student/production TypeScript stacks (114 scenarios): table drops, column removals, high-indegree file deletes.

Independent oracle. TypeScript import resolver + mark-down governance parser; no Neo4j reads. Column scenarios without symbol hits seed from table importers.

Agent. Claude Sonnet 4.5, editor role, temperature 0.

Conditions.

- **Enforcement**—prompt-only agent vs UCE gate (multi-repo).
- **MCP planning**—prompt-only vs MCP tools; *neutral* prompts (call tools, use output; no verbatim-copy instruction).
- **Context ladder**—bare LLM $\rightarrow$ +gov $\rightarrow$ +inventory $\rightarrow$ +RAG vs UCE graph (TalkAI).
- **RBAC hardness**—12-rule policy; 99 probes; Claude vs deterministic UCE.

Neo4j is ingested per repository before graph-backed conditions.

TABLE I
POOLED METRICS (INDEPENDENT FILE ORACLE).

Experiment	Recall	F1	Notes
Enforcement: agent self-catch	7.0%	—	114 scen., 4 repos
Enforcement: UCE gate catch	100%	—	0.9% false gates
Context+RAG (TalkAI)	44.2%	0.50	vs 84.2% UCE graph
RBAC: Claude breach	4.9%	—	UCE 0% breach

TABLE II
MCP AGENT PLANNING (NEUTRAL PROMPTS;
POOLED_SUMMARY.JSON).

Condition	Recall	F1	Incomplete
Prompt only	3.1%	0.06	100%
UCE MCP	98.2% [†]	0.95 [†]	12.5% [†]
MCP tool output	98.6%	—	—

[†]TalkAI $n=24$; multi-repo neutral run: `--prompt neutral --ingest`.

VI. RESULTS

Fig. 2 summarizes the three-layer findings.

A. Enforcement (primary)

Across 114 scenarios, agents self-catch 7.0% of issues (TalkAI 8.3%, Spark 2.1%). UCE gates fire on **100%** of scenarios with oracle-visible impact gaps; false gates: 1/114 (0.9%). Agents declare ~ 3.7 files mean vs UCE ~ 47 flagged (TalkAI median 67 missed). **Interpretation.** Prompt context cannot substitute for a gate—agents systematically under-specify transitive impact.

B. MCP planning (secondary, neutral prompts)

Table II reports pooled MCP metrics (profile tagged in `pooled_summary.json`). We report tool recall and adherence separately to avoid conflating correct tools with compliant agents.

Per-repo breakdown appears in Fig. 3 when multi-repo MCP completes.

C. Context ladder

RAG file recall ($\sim 44\%$) remains below UCE graph recall ($\sim 84\%$) on TalkAI—structured closure beats chunk retrieval (Fig. 4).

D. RBAC

Claude zero-shot breaches 4.9% of denied operations on RBAC_HARD_001; UCE deterministic evaluation: 0% breaches, 100% accuracy.

E. Ablation

Transitive import closure raises oracle-recall from 27% to 100%; call-chain edges add no gain on sparse CALLS (TalkAI)—reported as a negative result.

UCE evaluation summary

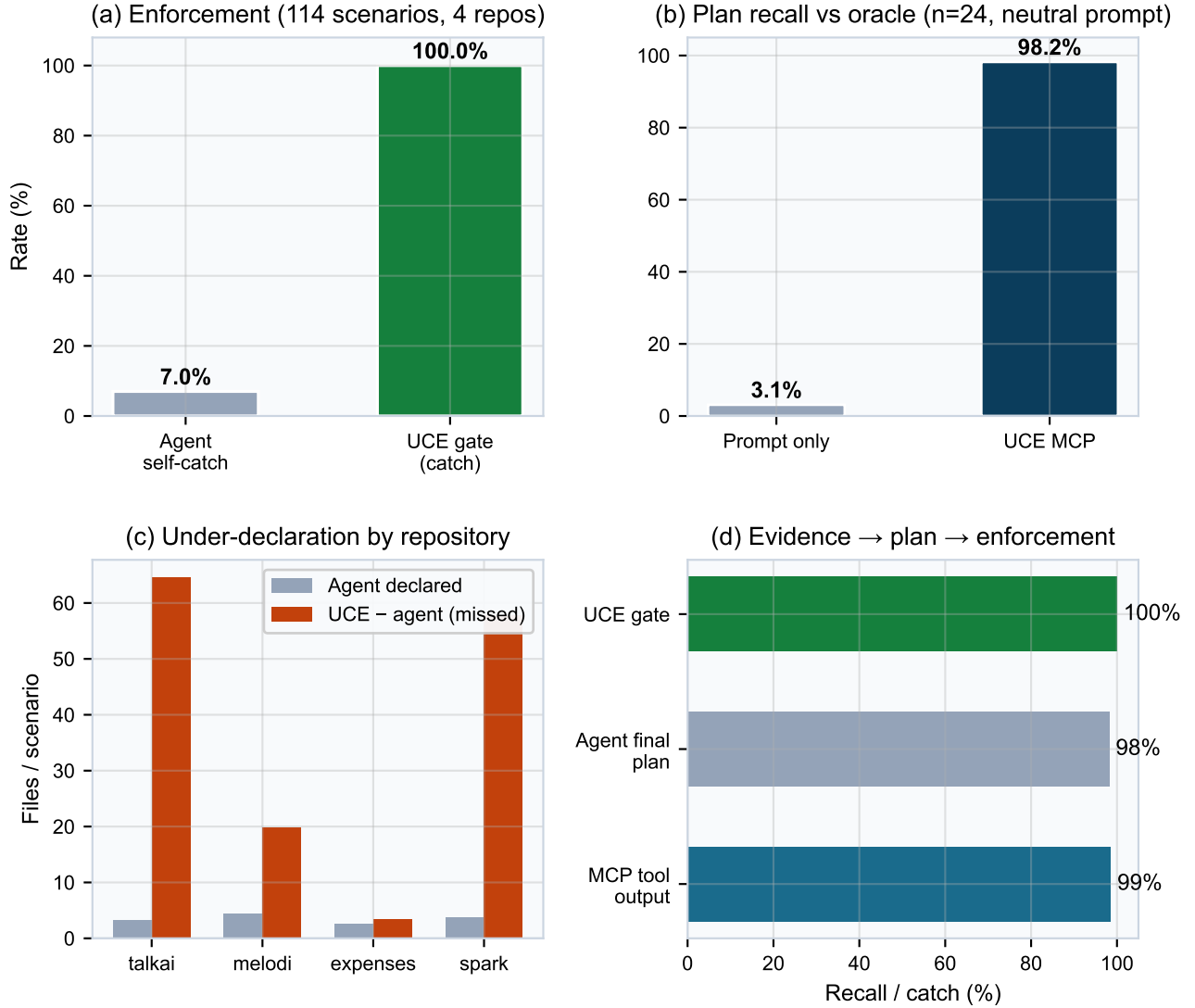


Fig. 2. Evaluation summary: (a) enforcement catch vs agent self-catch; (b) MCP plan recall (neutral prompts); (c) per-repo under-declaration; (d) tool output vs plan vs gate.

VII. THREATS TO VALIDITY

Oracle mismatch. Independent resolver may disagree with UCE on edge cases; we report madge agreement in supplementary retrieval studies.

Prompt sensitivity. Strict “copy verbatim” prompts inflate plan recall; we default to neutral prompts for MCP claims.

Agent/model scope. Single model family; costs limit scenario count.

Language scope. TypeScript-focused ingestion; generalization requires per-language resolvers.

VIII. DISCUSSION

For practitioners. Deploy UCE as MCP + gate: tools educate agents; gates protect when agents ignore tools.

For researchers. Report tool recall, plan recall, and gate catch separately—single F1 numbers hide deployment failure modes.

Limitations. UCE precision on heuristic `USES_TABLE` linking improved via import-gating but remains engineering-sensitive.

IX. CONCLUSION

UCE shows that governed agentic coding needs deterministic graph evidence and enforcement, not longer prompts. Multi-repo enforcement demonstrates near-universal agent under-specification; MCP tools supply correct blast radii; gates convert evidence into organizational safety.

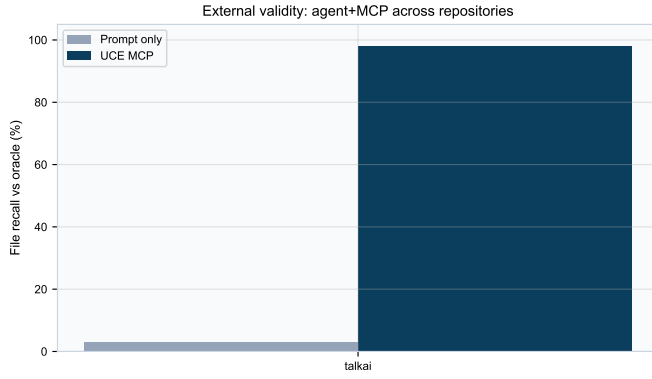


Fig. 3. File recall by repository (neutral MCP prompts).

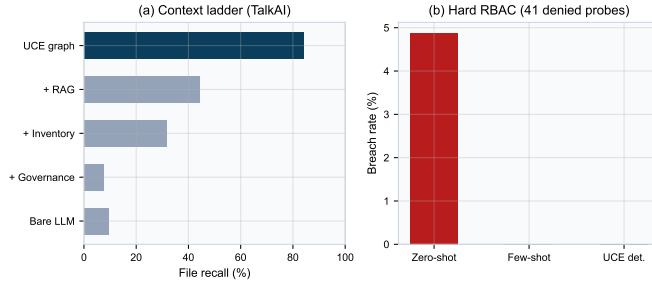


Fig. 4. Context ladder (left) and hard RBAC breaches (right).

X. IMPLEMENTATION NOTES

Each repository evaluation wipes and re-ingests Neo4j. The independent oracle shares alias conventions with ingestion but not code—disagreement is measurable in supplementary reports. MCP payloads surface `blast_radius_files` explicitly. Prompt profile `neutral` is default; `strict` (verbatim copy) is for ablation only.

REPRODUCIBILITY

Scripts: `run_multi_repo_enforcement.py`,
`run_agent_mcp_eval.py --prompt neutral`
`--ingest, run_context_comparison.py`,
`run_rbac_complexity.py`. Figures:
`paper/generate_figures.py`.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *International Conference on Learning Representations*, 2024.
- [2] J. Yang, C. E. Jimenez, A. Wettig *et al.*, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, 2024.
- [3] X. Liu, S. Gong, Y. Mao *et al.*, “Repobench: Benchmarking repository-level code auto-completion systems,” in *arXiv preprint arXiv:2306.03091*, 2023.
- [4] S. Yao, J. Zhao, D. Yu *et al.*, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [5] N. Shinn, F. Cassano, G. Berman *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023.

- [6] J. Wei, X. Wang, D. Schuurmans *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [7] Anthropic, “Model context protocol,” <https://modelcontextprotocol.io>, 2024.
- [8] D. Edge, D. Trampuz *et al.*, “From local to global: A graph RAG approach to query-focused summarization,” in *arXiv preprint arXiv:2404.16130*, 2024.
- [9] F. Liu *et al.*, “Codexgraph: Bridging large language models and code repositories via graph databases,” 2024.
- [10] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [11] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.